Traffic Simulation on Real Road Networks: Fixing, Improving, and Analyzing a ChatGPT-Generated Model

CS166, LBA

Contents

1 Introduction **2**

2 Bug Fixes **2**

3 Original Model and Its Visualizations **4**

4 Why the Original Model Is Insufficient **7**

5 Improved Model: Greenshields Density-Speed **7**

6 Limitations and Assumptions **9**

7 New Visualizations **10**

8 Empirical Congestion Analysis **13**

9 Network Metrics as Congestion Predictors **15**

10 Validation on a Different Location **17**

11 Bonus: Gridlock Across Network Topologies **21**

12 Conclusion **24**

Learning Outcomes **25**

AI Use Statement **26**

1 Introduction

This project starts with a ChatGPT-generated traffic simulation built on a real road network loaded from OpenStreetMap. The original notebook contained several bugs that prevented it from running, and its congestion model (a binary threshold that either allows or blocks all movement) fails to capture how real traffic behaves.

The goals of this project are to:

1. Fix the bugs and restructure the code into clean, documented classes.
2. Identify why the original model and its visualizations are insufficient.
3. Redesign the simulation using the Greenshields speed-density model, framed as a cellular automaton on a network graph.
4. Test whether network structure (edge betweenness centrality) can predict which roads become congested.
5. Investigate how network topology affects the onset of gridlock by comparing grid, radial, and scale-free networks.

The primary network is centered on Avenida Corrientes, Buenos Aires, with validation on the Plaza Italia / Palermo neighborhood. The gridlock analysis uses three synthetic networks to isolate the effect of topology.

2 Bug Fixes

2.1 Bugs Found and Fixed

The original ChatGPT notebook contained six bugs. Each is described below with the fix.

Bug 1: No Path Between Random Nodes. The road network loaded from OpenStreetMap is a *directed* graph (many streets are one-way), so a randomly chosen start/destination pair may have no directed path between them, raising a `NetworkXNoPath` exception. The fix is to extract the largest strongly connected component (the maximal subgraph in which every node can reach every other node) before placing any cars:

```
graph = ox.utils_graph.get_largest_component(graph, strongly=True)
```

Bug 2: path.pop(0) Removes the Wrong Node. The path-following logic called `path.pop(0)` to get the next node, but `nx.shortest_path()` returns a list that *starts with the source node*. Popping index 0 removes the car's current location rather than advancing it. The fix is to slice off the source immediately after computing the path:

```
path = nx.shortest_path(graph, source, target, weight='travel_time')[1:]
```

Bug 3: Simulation Never Returns Its Results. The simulation function stored its car list in a local variable and never returned it, so the visualization cell raised a `NameError`. Converting the simulation into a class method that returns `self` solved this scoping issue:

```
def run(self):
    # ... simulation logic ...
    return self
```

Bug 4: Edge Lookup Uses Wrong Key Type. The heatmap code looked up edge traffic counts using `data['osmid']` as the dictionary key. However, OSMnx represents the road network as a `MultiDiGraph` where edges are keyed by `(u, v, k)` tuples, not by OSM identifiers. Every lookup silently returned the default value of 0, producing a blank heatmap. The fix is to iterate using the graph's actual edge keys:

```
for u, v, k in graph.edges(keys=True):
    count = edge_counts.get((u, v, k), 0)
```

Bug 5: Graph Loaded Three Separate Times. Three different cells each called `ox.graph_from_address()`, wasting API calls and causing inconsistency (only one copy had `travel_time` attributes added). The fix was to load once and reuse the same graph object everywhere.

Bug 6: Division by Zero in Color Mapping. When all cars have reached their destinations, every edge count is zero, so `max(counts.values())` returns 0 and the color normalization divides by zero. The fix is a one-line guard:

```
max_count = max(max(counts.values()), 1)
```

2.2 OOP Restructuring

The original code used loose functions and global variables. It was restructured into three classes with docstrings:

- **RoadNetwork**: loads the graph, extracts the strongly connected component, adds travel time attributes, and plots the network.
- **Car**: stores start, destination, path, and provides a `has_arrived` property.
- **TrafficSimulation**: holds the network and cars, runs the movement logic, tracks congestion history, and provides visualization methods.

3 Original Model and Its Visualizations

3.1 How the Original Model Works

After bug fixes, the simulation works as follows:

1. N cars (default 100) are placed at random nodes with random destinations.
2. Each time step, each car computes a shortest path via Dijkstra's algorithm (a graph search algorithm that finds the shortest weighted path between two nodes) using `travel_time` edge weights.
3. Before moving, the car counts how many cars (including itself) are at the same node and trying to move to the same next node.
4. If that count exceeds the threshold (default 5, so 6+ cars total), the car waits.
5. Otherwise, the car advances one node along its path.

The simulation runs for a fixed number of steps (default 10).

3.2 Original Visualizations

Three visualizations were added to the bug-fixed simulation. Although these figures color *edges*, the underlying data is entirely node-based. Cars exist only at nodes, and congestion is determined by counting cars at the same node competing for the same next node. The edge colors simply reflect how many times a car jumped from one node to the next across that edge. Section 4 explains why this is a fundamental limitation.

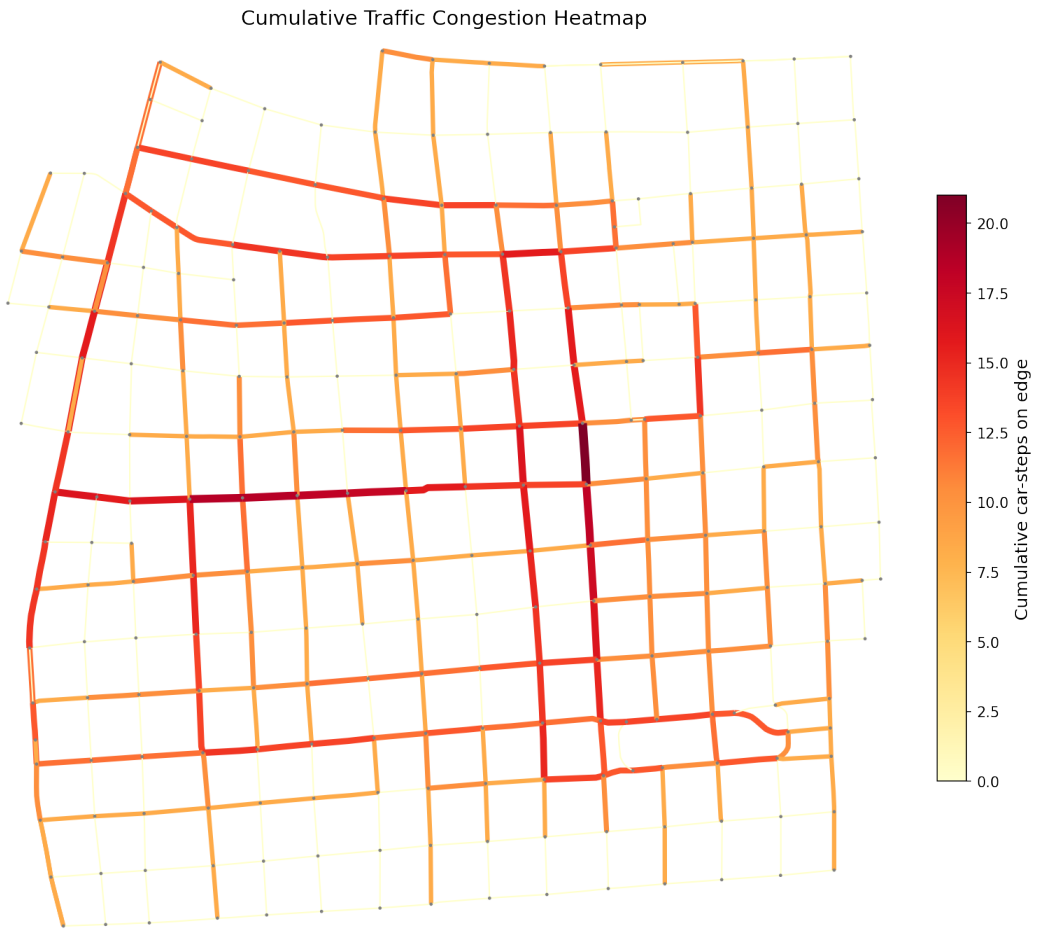

Figure 1: Cumulative congestion heatmap of the Avenida Corrientes network after running 100 cars for 10 steps using the original binary threshold model. Edge color (yellow to dark red) and width scale with total car-steps accumulated over the simulation. Although the visualization colors edges, the underlying congestion check is node-based: cars exist only at nodes, and the count reflects node-to-node jumps, not edge-level density. This measures traffic *volume*, not actual congestion.

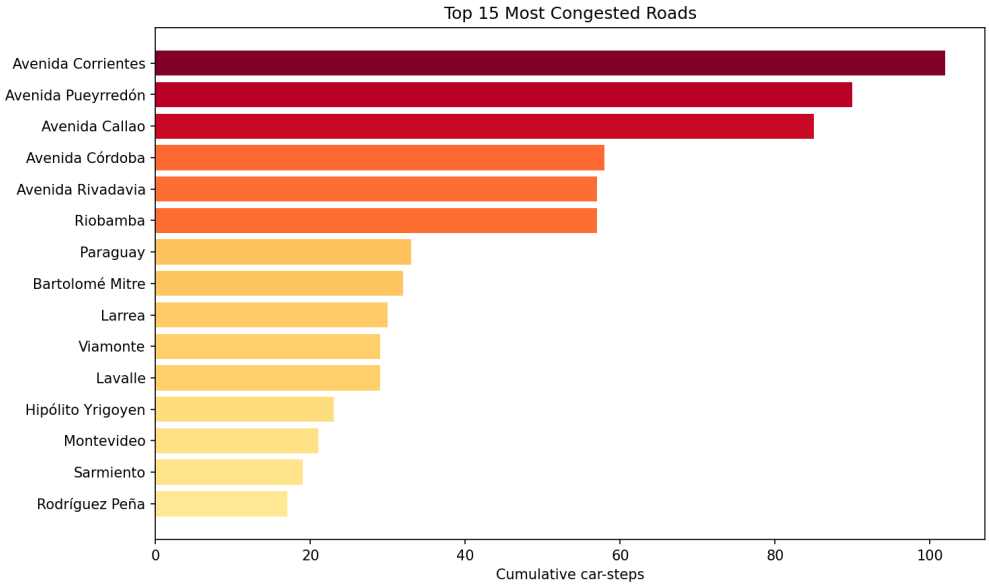

Figure 2: Top 15 streets in the Corrientes network ranked by cumulative car-steps from the original model. Streets composed of multiple edge segments are aggregated by name so the full road appears as a single bar. Because the model only tracks cars at nodes, these counts reflect route popularity (how many cars passed through), not congestion. A road with steady flow and a road approaching gridlock can produce similar counts.

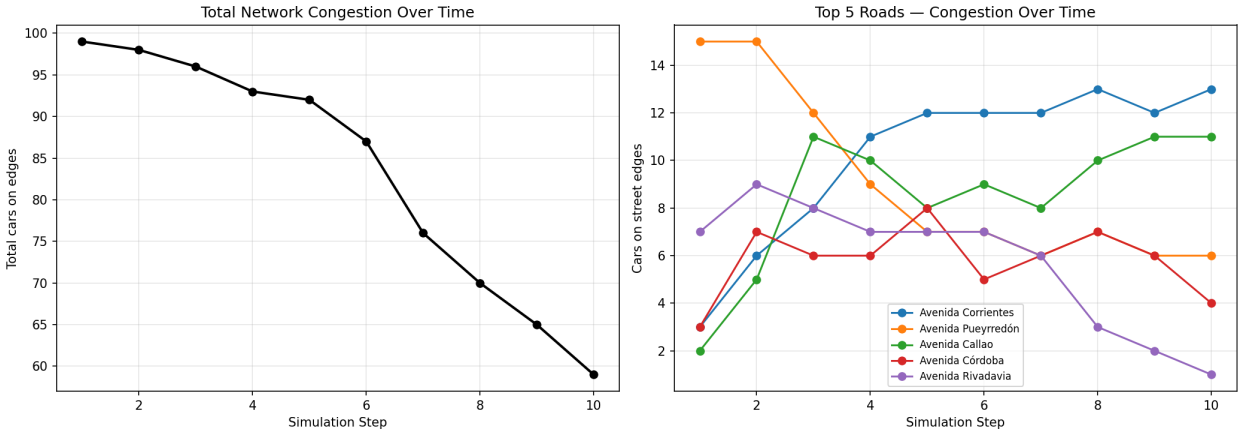

Figure 3: Congestion dynamics over 10 simulation steps. Left panel: total cars still in transit across the network at each step, where the falling line shows cars reaching their destinations. Right panel: per-step car counts on the top 5 most-used streets. Because the model only checks congestion at nodes, these counts reflect how many cars used each street per step, not how congested the road actually was.

3.3 What These Visualizations Actually Show

These visualizations show traffic *volume* (where cars went), not *congestion* (where cars slowed down). "Cumulative car-steps" counts successful node-to-node traversals; it does not mea-

sure delay, slowdown, or density. A road with 1 car every step and a road with 5 cars every step look different in volume, but both are flowing freely under the threshold. The next section explains the structural reasons for this limitation.

4 Why the Original Model Is Insufficient

4.1 Structural Problems

- **Binary congestion.** Below the threshold, cars move at full speed. Above it, they stop completely. Real traffic slows down gradually as density increases.
- **Arbitrary threshold.** The threshold of 5 cars has no relationship to road properties. A narrow residential street and a 6-lane avenida have the same capacity.
- **No position tracking.** Cars exist only at nodes, not along edges. A 50m alley and a 500m avenue process cars at the same rate (one node per step). Edge length is ignored entirely, and density (cars/meter) cannot be computed.

4.2 Why This Produces Misleading Results

Because congestion is binary, the visualizations can only show where cars *went*, not where they *slowed down*. Edges that appear "congested" in the heatmap may simply be popular routes with no actual delay. The model cannot distinguish between a busy road flowing smoothly and a gridlocked road, since both are invisible to the threshold check as long as they stay under 6 cars.

5 Improved Model: Greenshields Density-Speed

5.1 The Greenshields Model

We needed a model that incorporates road-level density (cars per meter) and converts it into a continuous speed. The Greenshields speed-density relationship (Greenshields, 1935) does exactly this with a single linear equation whose two parameters, v_{free} (speed limit) and ρ_{jam} (jam density from lane count), are both already available in the OpenStreetMap data. No calibration is required, and because our analysis focuses on *relative* congestion patterns rather than exact travel times, the linear approximation is sufficient. The relationship is:

$$v = v_{\text{free}} \times \max\left(1 - \frac{\rho}{\rho_{\text{jam}}},\ 0.05\right) \tag{1}$$

where v_{free} is the free-flow speed (the posted speed limit from OSM data, i.e. the speed a car travels when alone on the road), ρ is the current density (cars/meter on the edge), and ρ_{jam} is the jam density (the maximum number of cars per meter that can physically fit on the road, determined by the number of lanes, so more lanes means more cars can pack in before the road is fully jammed). In our implementation, $\rho_{\text{jam}} = 0.04 \times \text{lanes}$ cars/meter;

the standard traffic engineering value is ∼0.15 cars/m/lane (∼one car every 7 m), but we scale it down so that congestion effects are visible with the simulation's relatively small car counts. Speed decreases continuously as density increases: at zero density, cars travel at free-flow speed; at jam density, they slow to 5% of free-flow speed (the floor prevents cars from fully stopping, which would deadlock the simulation).

5.2 Implementation

The improved model is implemented via two subclasses that inherit from the original classes, overriding only the movement logic:

- **ImprovedCar(Car)**: adds `position` (meters along the current edge), `current_edge`, and `speed` attributes.
- **ImprovedTrafficSimulation(TrafficSimulation)**: overrides `_move_cars()` to implement Greenshields:
  1. Cars are spawned at random positions along random edges (not just at nodes).
  2. Each step: count cars per edge, compute density $\rho = \text{cars/edge_length}$.
  3. Derive speed from Equation 1.
  4. Advance each car's position by its speed. When position exceeds the edge length, the car transitions to the next edge.
  5. Jam density ρ_{jam} varies by lane count from OSM data.

Each time step represents approximately 6 seconds; free-flow speed at 30 km/h is approximately 50 m/step. All visualization methods are inherited from the parent class.

5.3 What This Fixes

This model directly addresses each structural problem identified in Section 4:

- **Binary congestion → continuous.** Instead of a hard threshold where cars either move at full speed or stop entirely, the Greenshields equation produces a smooth speed curve. A road with 3 cars behaves differently from a road with 10 cars, which behaves differently from a road with 50 cars.
- **Arbitrary threshold → physical parameters.** The original model's threshold of 5 had no connection to road properties. Greenshields replaces it with ρ_{jam}, which varies by lane count: a 3-lane avenue can hold three times as many cars per meter as a single-lane street before reaching jam density.
- **No position tracking → edge-level density.** Cars now have positions along edges (in meters), so density can be computed as cars per meter. A 50 m alley with 2 cars is much more congested than a 500 m avenue with 2 cars, and the model captures this difference.

5.4 Connection to Cellular Automata

The improved simulation can be understood as a cellular automaton (CA) operating on a network graph. Each edge is a "cell" whose state is its current density (cars/meter). The update rule is the Greenshields equation (Equation 1): at each discrete time step, each edge's density determines the speed of its cars, which in turn determines how many cars leave the edge and enter adjacent edges in the next step. The state of each cell at time $t + 1$ depends on its own state and the states of its neighbors at time t, which is the defining property of a cellular automaton.

This contrasts with the Nagel-Schreckenberg (NaSch) model (Nagel and Schreckenberg, 1992), the canonical traffic CA. NaSch operates on a one-dimensional lattice where each cell is a road position and the state is "car or empty." Each car follows four micro-rules per step: accelerate (up to a max speed), brake (if the car ahead is too close), randomly decelerate (modeling human unpredictability), and move forward. Our model operates on a network graph where cells are edges and the state is a continuous density value, updated by a single macro-rule (Greenshields).

Both are cellular automata, but at different scales. NaSch is *microscopic*: it tracks individual car decisions (accelerate, brake, hesitate). Our model is *macroscopic*: it tracks aggregate density and derives speed from it. The microscopic approach captures phenomena like phantom traffic jams (spontaneous stop-and-go waves), while the macroscopic approach captures how network topology distributes and concentrates congestion. Section 11 uses this framing to compare how different graph topologies handle increasing traffic.

6 Limitations and Assumptions

The improved model addresses the structural problems of the original, but it introduces its own simplifications. Each simplification is a trade-off, but none of them affect the relative congestion patterns that our analysis depends on.

Linear speed-density relationship. The Greenshields model assumes that speed decreases *linearly* with density. In reality, the speed-density relationship is nonlinear; alternative models such as Greenberg (logarithmic) and Underwood (exponential) better capture the sharp speed drop near jam density. We chose Greenshields because it has only two parameters (v_{free} and ρ_{jam}), is analytically transparent, and produces qualitatively correct behavior: free flow at low density, gradual slowdown, and gridlock at jam density. Since our analysis focuses on *relative* congestion patterns (which edges congest more than others) rather than precise speed values, the linear approximation is sufficient.

Static routing. Cars compute their shortest path once at spawn and follow it to their destination, regardless of congestion encountered along the way. Real drivers adapt: they check navigation apps, take side streets, or avoid visibly jammed roads. This means our simulation overestimates congestion on shortest-path bottlenecks and underestimates load on alternative routes. The effect is especially relevant for the grid topology analysis (Section 11), where the grid's many parallel paths would become more useful under dynamic rerouting.

No intersection delay. Cars transition instantly between edges at nodes. Real intersections impose delays: traffic lights, stop signs, turning conflicts, and pedestrian crossings all add time that depends on intersection geometry and signal timing. This simplification means our simulation underestimates total travel time and cannot capture intersection-specific bottlenecks. However, since we compare congestion *across edges* rather than predicting absolute travel times, the omission is acceptable for our purposes.

Uniform car behavior. All cars follow the same Greenshields rule with identical parameters. There is no variation in driver aggression, vehicle size, reaction time, or willingness to take longer routes. This is a consequence of the macroscopic modeling choice: individual car heterogeneity is averaged out in favor of aggregate density dynamics. A microscopic model like Nagel-Schreckenberg would capture these differences but at much greater computational cost and complexity.

7 New Visualizations

7.1 Car Animation

An animated visualization shows cars (blue dots) moving through the network in real time. Cars visibly slow down on congested edges and flow freely on empty ones, directly demonstrating the Greenshields model. Figure 4 shows a representative frame.

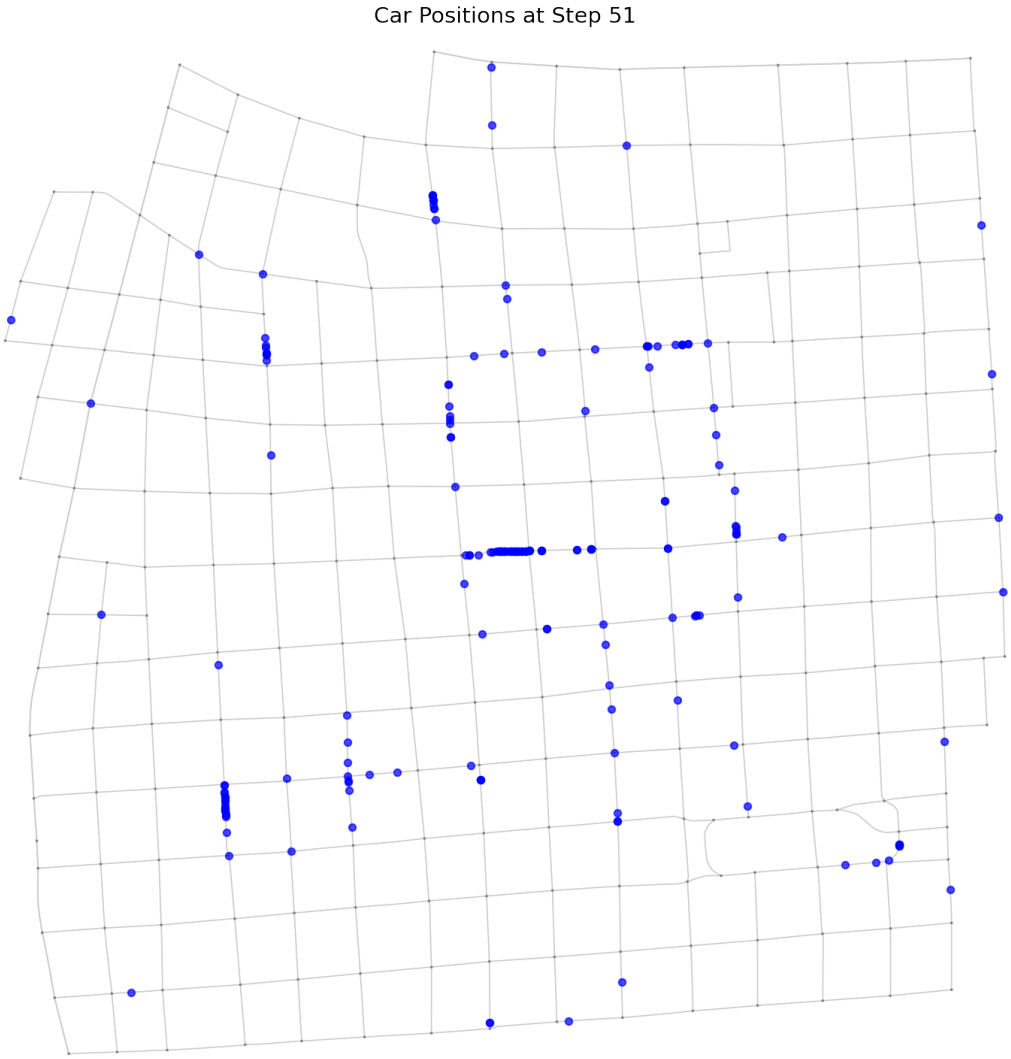

Figure 4: Representative frame from the car animation at step 51 of 100, showing 500 cars (blue dots) distributed across the Corrientes road network. Car positions are interpolated along edges based on their Greenshields-derived speeds. Clusters of dots on certain edges indicate congestion, where cars are slowing down due to high local density. The full interactive animation with play/pause controls is available in the notebook.

7.2 Density Heatmap Animation

Edge colors show local density (cars/meter) evolving over time. Congestion builds at bottleneck edges and dissipates as cars clear. Figure 5 shows a representative frame.

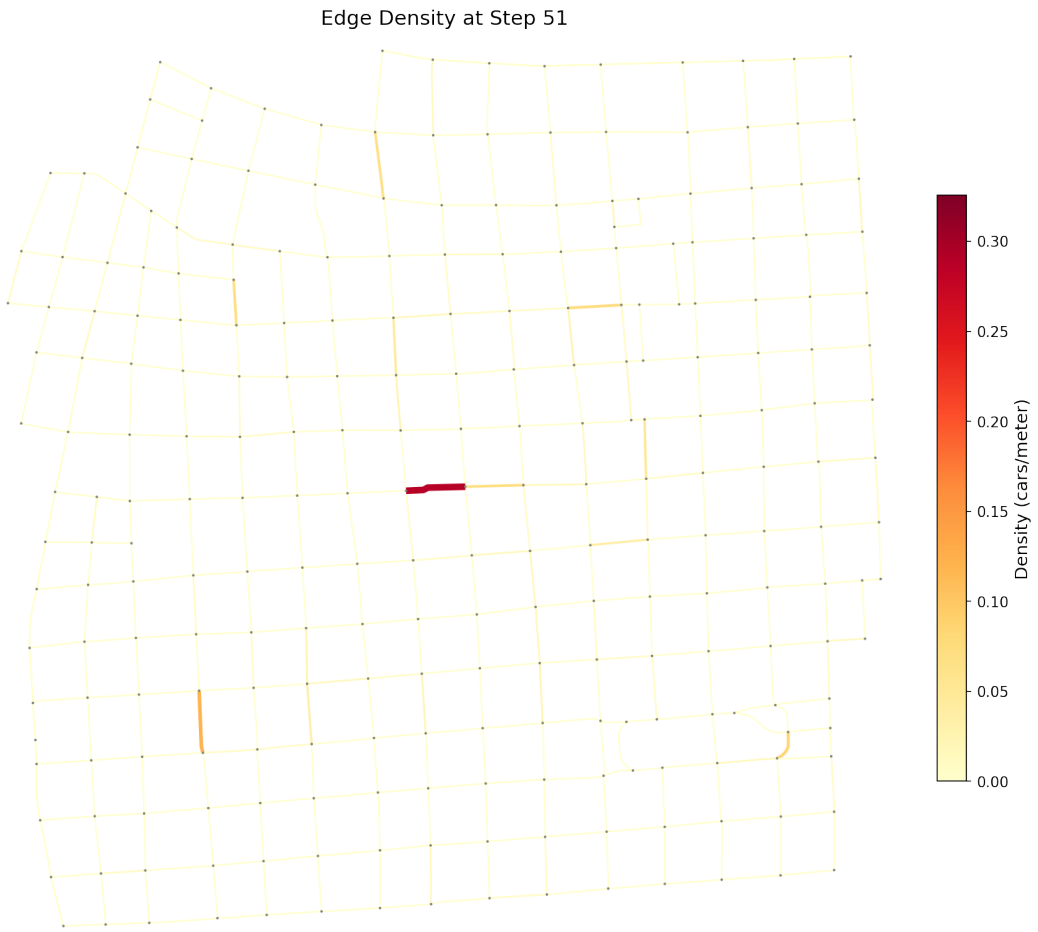

Figure 5: Representative frame from the density heatmap animation at step 51 of 100. Each edge is colored by its instantaneous density (cars/meter): yellow indicates low or zero density, dark red indicates high density approaching jam conditions. Edge widths also scale with density. The colorbar provides the quantitative mapping. The full animation shows congestion building at bottlenecks and dissipating as cars clear.

7.3 Average Density Heatmap

A static summary of the average density per edge across all simulation steps (Figure 6). This is the congestion measure used in the subsequent multi-run analysis.

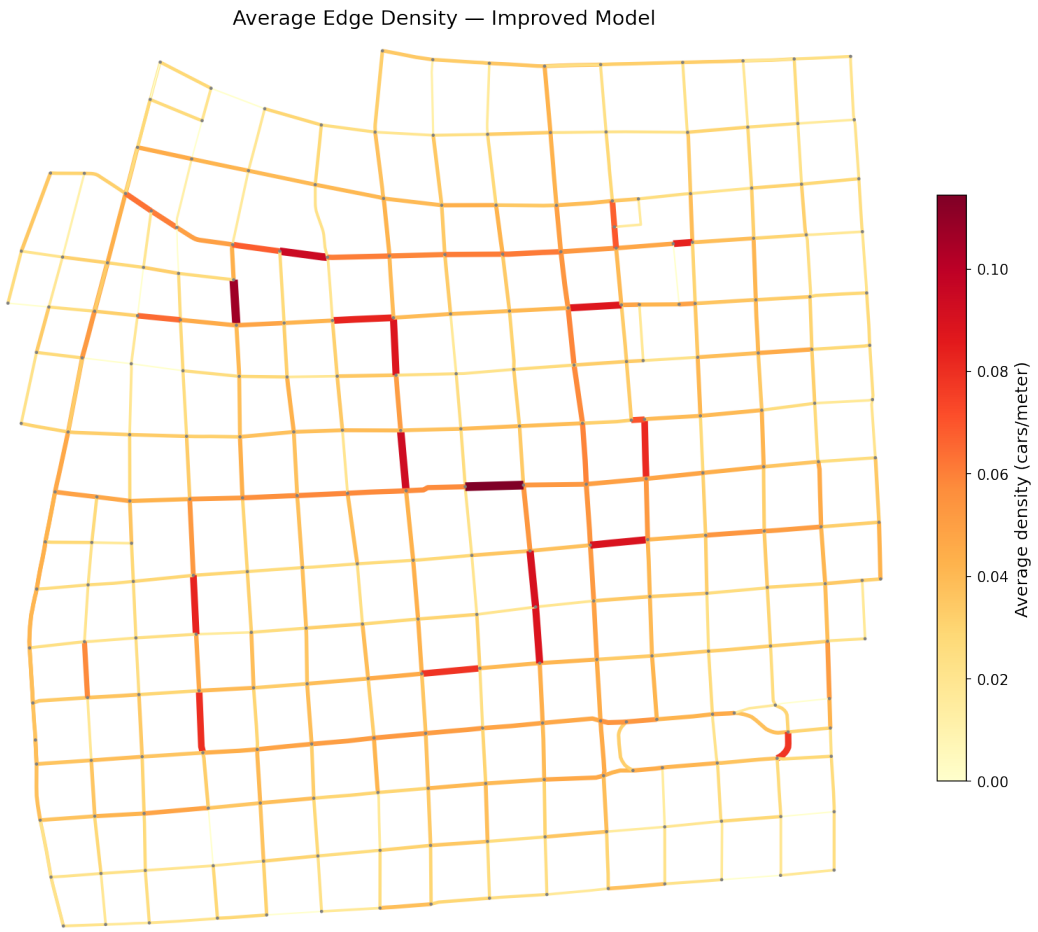

Figure 6: Average edge density (cars/meter) over a single simulation run with 500 cars and 100 steps using the Greenshields model. Unlike the original model's cumulative car-steps, density accounts for road length. A short edge with few cars can show higher density than a long edge with many cars. The colorbar maps edge color to average density. This is the congestion measure used in the subsequent multi-run analysis.

7.4 Comparison: Original vs Improved

The original visualizations (Section 3) show where cars *went*. These new visualizations show where cars actually slowed down.

8 Empirical Congestion Analysis

8.1 Choosing the Number of Runs: Convergence Analysis

A single simulation run is noisy; random car placements mean different roads may appear congested for incidental rather than structural reasons. To determine how many independent runs are needed before the average edge density estimate stabilizes, we track the mean absolute change in per-edge average density after each successive run:

$$\delta_k = \frac{1}{|E|} \sum_{e \in E} \left| \bar{\rho}_e^{(k)} - \bar{\rho}_e^{(k-1)} \right|$$

where $\bar{\rho}_e^{(k)}$ is the cumulative average density on edge e after k runs. When δ_k is near zero, additional runs contribute negligible information.

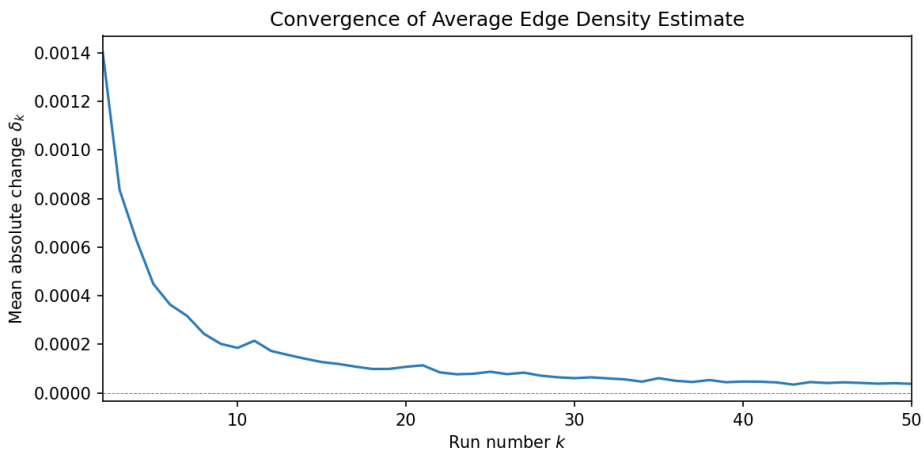

Figure 7: Convergence of the average edge density estimate as a function of run count. The mean absolute change δ_k drops sharply in the first 10–15 runs and flattens thereafter, confirming that 50 runs is sufficient for a stable estimate.

As Figure 7 shows, δ_k falls sharply in the first 10–15 runs and then levels off, confirming that 50 runs is sufficient. This justifies our ensemble size empirically rather than by arbitrary choice.

8.2 Methodology

With 50 runs confirmed as sufficient by the convergence analysis above, we use this ensemble size for the full empirical study. Each of the 50 independent simulations runs the improved Greenshields model with 500 cars for 100 steps, and we average the density per edge across all runs. Edges that consistently show high density across this ensemble are true structural bottlenecks, not artifacts of any single random placement.

8.3 Results

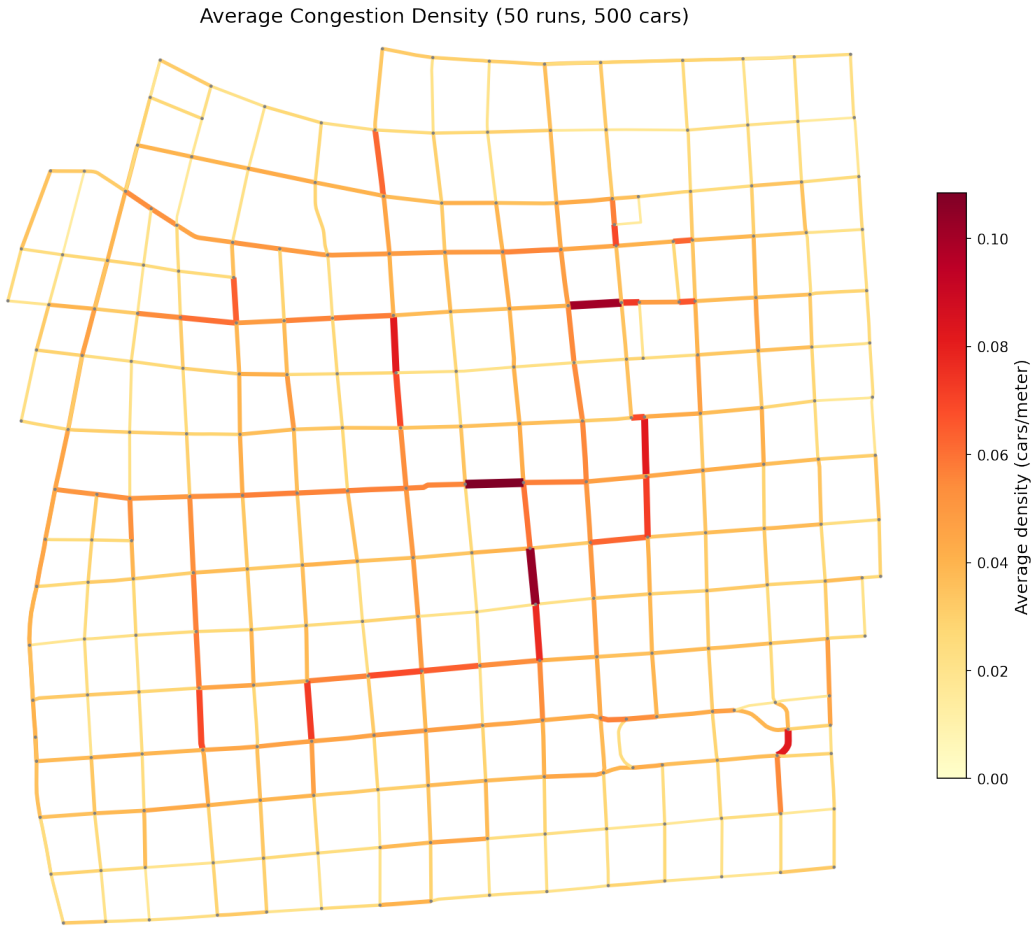

Figure 8: Average congestion density (cars/meter) across 50 independent simulation runs, each with 500 cars and 100 steps. Averaging removes noise from random car placements and reveals edges that are *structurally* prone to congestion due to their position in the network. Darker red and thicker edges indicate consistently higher density. The pattern is stable across runs, confirming these are true bottlenecks rather than random artifacts.

9 Network Metrics as Congestion Predictors

9.1 Why Edge Betweenness Centrality

Most standard network metrics (degree centrality, closeness, clustering coefficient, etc.) are defined at the *node* level. However, congestion happens on *edges* (road segments between intersections), and mapping node metrics to edges by averaging endpoints is not principled.

Edge betweenness centrality is one of the few metrics defined directly on edges. It measures the fraction of all shortest paths in the network that traverse a given edge. Since our cars route via shortest paths, this is a direct structural prediction of traffic demand.

Edges on many shortest paths should attract more cars and therefore experience higher density.

9.2 Correlation Analysis

We compute edge betweenness centrality (weighted by `travel_time` to match how cars route) and correlate it against the empirical density from the multi-run analysis. We use the Spearman rank correlation (Spearman, 1904) rather than Pearson because the relationship between betweenness and density is not linear (as visible in the scatter plot). Spearman measures whether the rankings agree, which is what we care about: can betweenness predict *which* edges congest? The result is $r = 0.88$ ($p < 10^{-160}$).

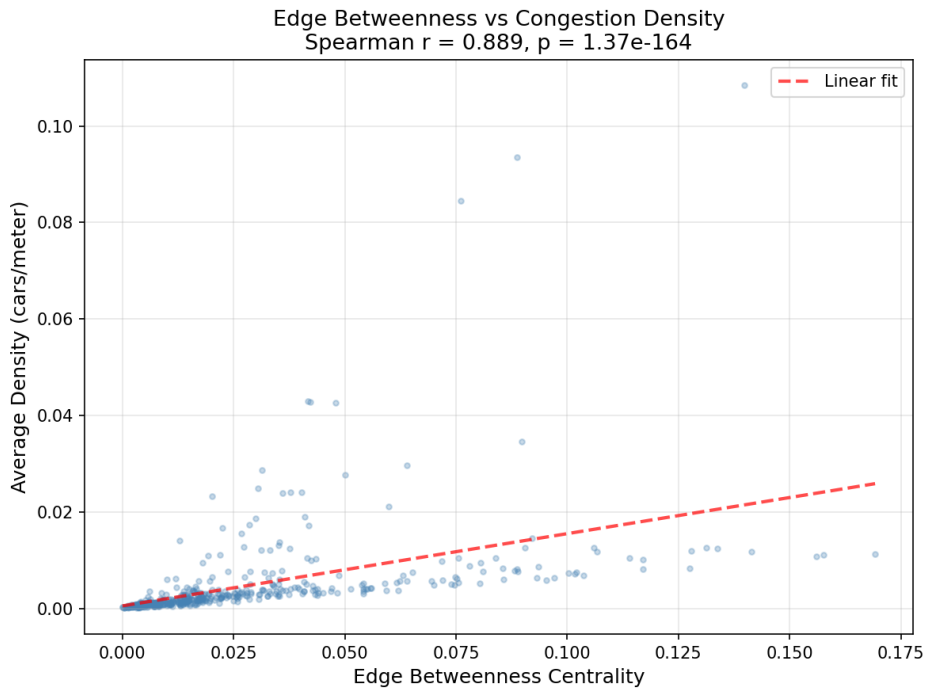

Figure 9: Scatter plot comparing edge betweenness centrality (x-axis) to empirical average density from 50 simulation runs (y-axis) on the Corrientes network. Each point represents one edge. The strong positive correlation (Spearman $r = 0.88$, $p < 10^{-160}$) confirms that edges lying on many shortest paths experience higher congestion density. The red dashed line shows the linear trend.

9.3 Visual Comparison

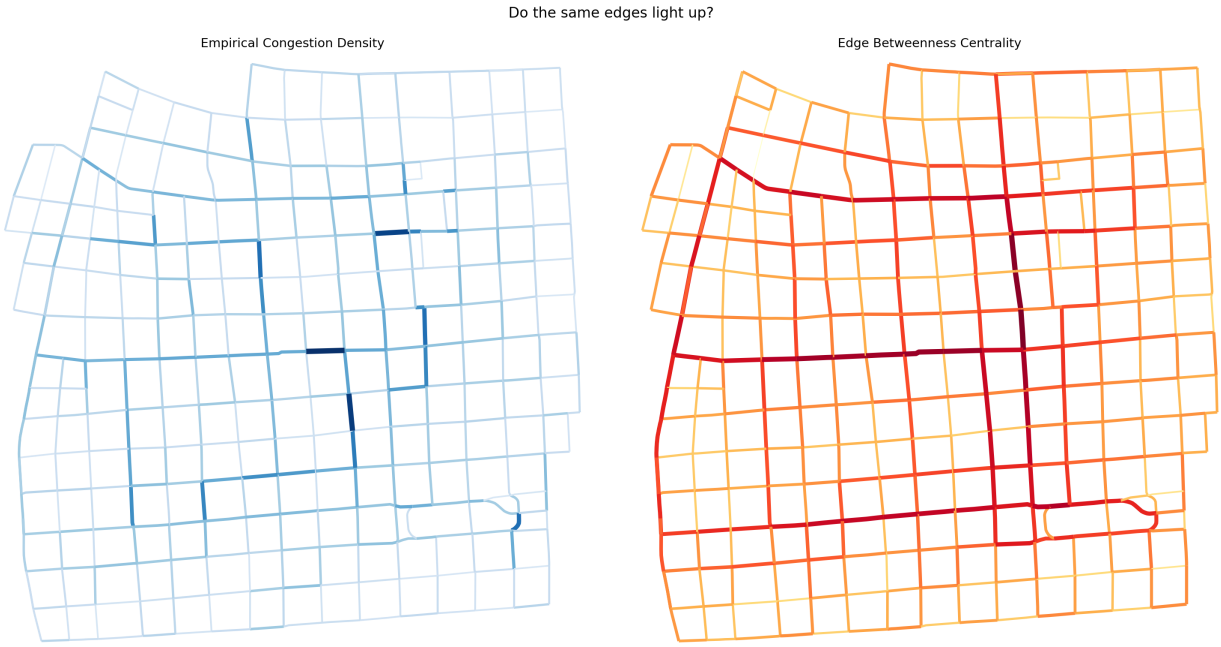

Figure 10: Side-by-side spatial comparison on the Corrientes network. Left: empirical congestion density from 50 simulation runs. Right: edge betweenness centrality computed purely from network structure. The empirical map uses a Blues color scale and the betweenness map uses YlOrRd, so they can be compared without conflating the two measures. The highest-traffic edges in the empirical map generally correspond to the highest-betweenness edges, though the betweenness map highlights more edges overall since it captures potential rather than realized traffic.

The two maps in Figure 10 show broad spatial agreement: the edges with the highest empirical density tend to also have high betweenness centrality. The match is not perfect (some high-betweenness edges show only moderate density, and a few high-density edges are not the top-ranked by betweenness), but the overall pattern is consistent with the Spearman $r = 0.88$.

10 Validation on a Different Location

10.1 Validation Setup

To test whether edge betweenness centrality generalizes as a congestion predictor, we repeat the analysis on a second road network: Plaza Italia / Palermo, Buenos Aires. This neighborhood has a different road topology from the Corrientes downtown area. The same methodology is used: 50 simulation runs, 500 cars, 100 steps.

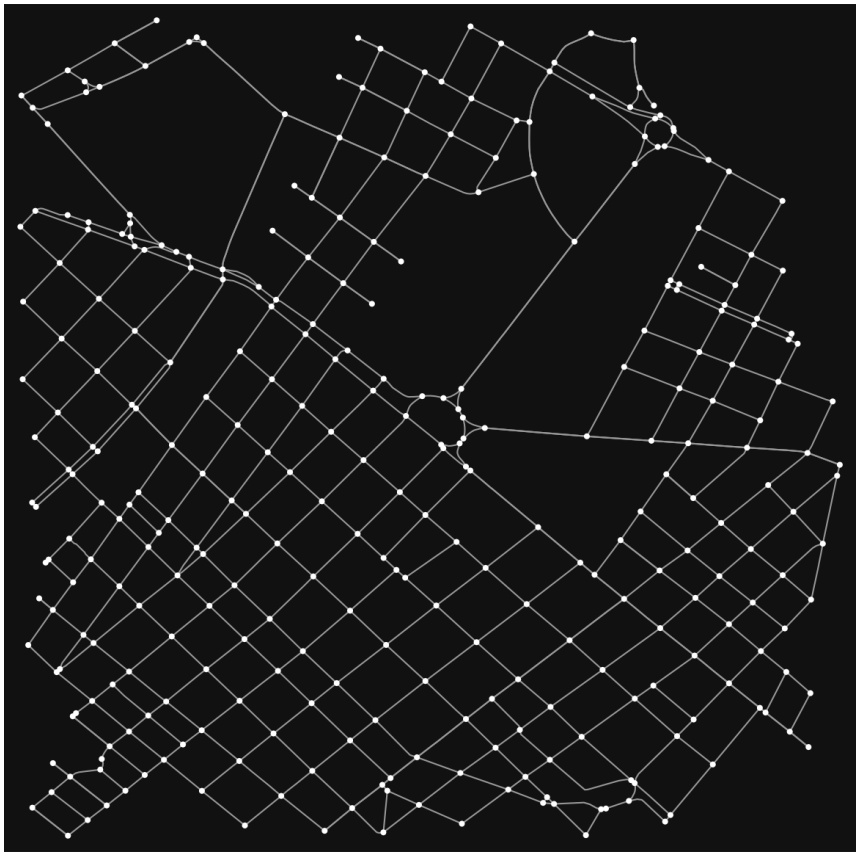

Figure 11: The Plaza Italia / Palermo road network used for validation, loaded from OpenStreetMap with a 1 km radius. This neighborhood has a different topology from the downtown Corrientes area, with wider boulevards, more parks, and a different grid structure. The network is reduced to its largest strongly connected component to ensure full reachability.

10.2 Results

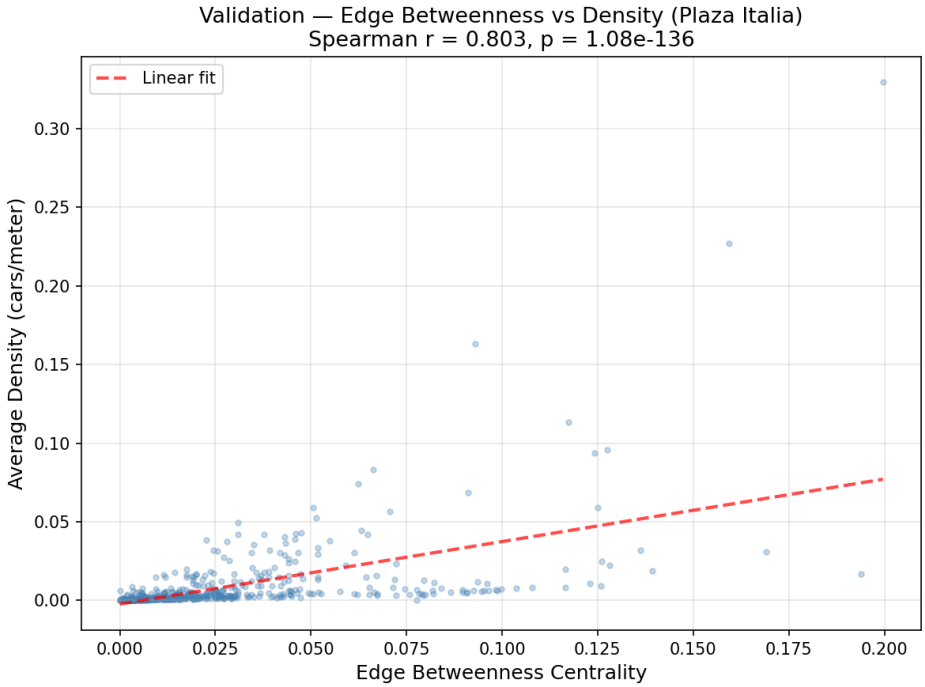

Figure 12: Scatter plot of edge betweenness centrality vs. empirical density on the Plaza Italia validation network ($r = 0.81$, $p < 10^{-139}$). The positive correlation persists on this different topology, though slightly weaker than the Corrientes network ($r = 0.88$). This confirms that edge betweenness generalizes as a congestion predictor across different road network structures.

Figure 13: Side-by-side spatial comparison on the Plaza Italia validation network. Left: empirical congestion density averaged across 50 runs (Blues scale). Right: edge betweenness centrality from network structure alone (YlOrRd scale). The general trend holds (high-betweenness edges tend to also show higher empirical density), though the match is noisier than on the Corrientes network, consistent with the lower Spearman $r = 0.81$.

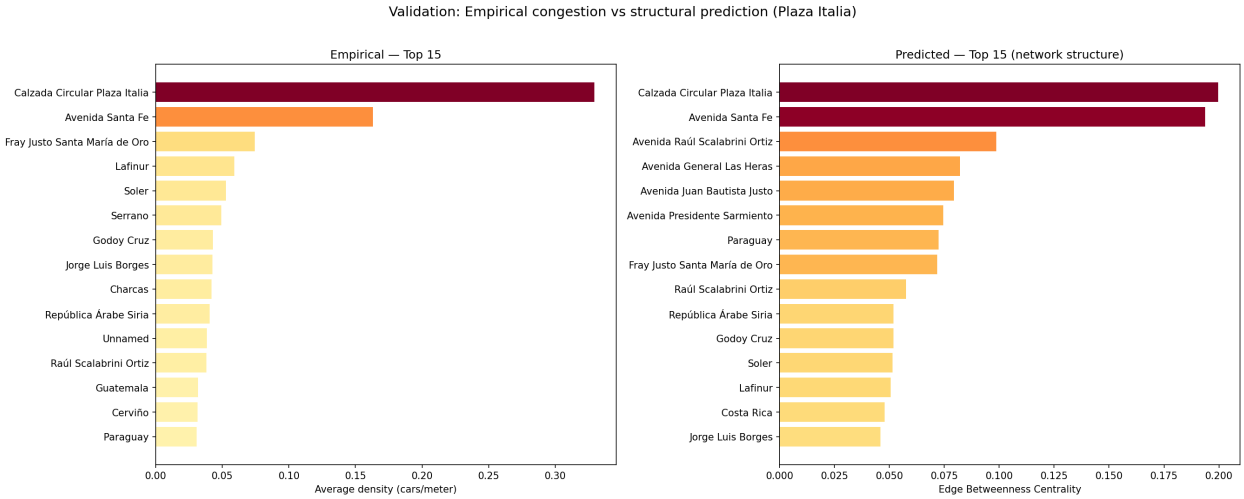

Figure 14: Side-by-side comparison of the top 15 streets in the Plaza Italia validation network. Left: ranked by empirical congestion density (cars/meter) from 50 simulation runs. Right: ranked by edge betweenness centrality from network structure. Six streets appear in both lists (Calzada Circular Plaza Italia, Psje punto Santa María de Oro, Raúl Scalabrini Ortíz, República Árabe Siria, Jorge Luis Borges, and Latitud), confirming partial but meaningful overlap between empirical and structural rankings.

10.3 Discussion

Edge betweenness centrality remains a strong predictor of empirical congestion density on the Plaza Italia network, with a Spearman correlation of $r = 0.81$ ($p < 10^{-139}$). This is slightly lower than the Corrientes result ($r = 0.88$), which is expected: the Palermo area has a less grid-like topology with more irregular intersections, so the relationship between shortest-path structure and congestion is noisier. Nevertheless, the correlation is still strong and highly significant.

The side-by-side bar chart (Figure 14) shows substantial overlap between the streets ranked highest by empirical density and those ranked highest by edge betweenness. Streets that serve as connectors between major intersections appear in both lists, confirming that the structural prediction is not specific to the Corrientes network.

Since the prediction works on a network it was never calibrated on, the result is likely general: shortest-path routing concentrates traffic on high-betweenness edges regardless of the specific network layout. The slight decrease in correlation from Corrientes to Palermo suggests that network regularity (grid structure) strengthens the prediction, but even irregular networks exhibit the relationship clearly.

11 Bonus: Gridlock Across Network Topologies

11.1 Motivation

The previous sections showed that edge betweenness centrality predicts congestion on real road networks. But real road networks have a specific topology (roughly grid-like). A core question in network science is whether the same local update rule can produce different macro-level behavior depending on the structure of the system. Congestion on one road does not stay local; it propagates through the network as upstream edges back up and downstream edges starve. How far and how fast this propagation occurs depends on how the network is connected: a city built around a single central artery will experience network-wide gridlock more readily than one with many redundant paths. To test this, we apply the same Greenshields CA update rule to three synthetic network topologies and ask: how does congestion scale with the number of cars, and which topologies are most vulnerable to network-wide breakdown?

11.2 Methodology

We generate three synthetic graphs with approximately the same number of nodes (140–150) but different connectivity structures. All edges have uniform properties (length = 100 m, speed limit = 30 km/h, lanes = 1) so that topology is the only variable. The three graphs are also constructed with similar node and edge counts (133–150 nodes, 504–592 directed edges) so that differences in the speed curves reflect topology, not network size. This is a deliberate simplification: real cities have varying road widths, speed limits, and network sizes, but uniform properties isolate the structural effect.

For each topology, we sweep the number of cars from 25 to 1000 in 11 steps. At each car count, we run 20 independent simulations of 100 steps each. After each step, we record the

average speed fraction across all active cars: each car's speed divided by the free-flow speed on its current edge, so 1.0 means free flow and 0.0 means stopped. We report steady-state values: the average speed fraction computed over only the last 20% of simulation steps, by which point the initial transient (cars spreading out from random starting positions) has settled and the network has reached a roughly stable congestion level. We also report the standard error of the mean across runs, which is appropriate because we are estimating the expected average at each car count, not the variability of individual runs.

11.3 Graph Topologies

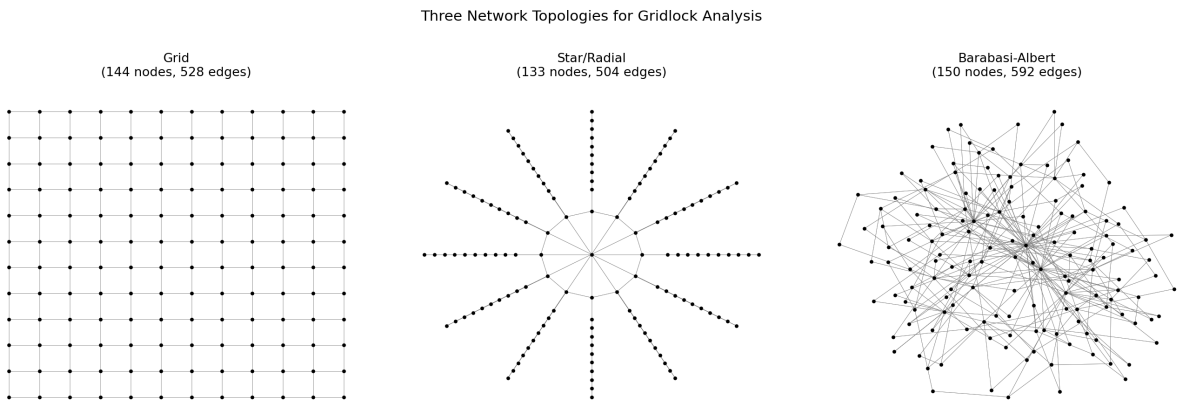

Figure 15: The three synthetic network topologies used for gridlock analysis. Left: a 12×12 grid representing a Manhattan-style city with high path redundancy. Center: a hub-and-spoke radial network where cross-sector traffic funnels through a central hub. Right: a Barabási-Albert scale-free network where a few high-degree hubs attract disproportionate traffic. All edges have identical properties (100 m, 30 km/h, 1 lane) so topology is the only variable.

- **Grid** (144 nodes, 528 directed edges): A 12×12 grid with bidirectional edges. Every node has degree 4 (or less at boundaries). Represents Manhattan-style cities with regular, uniform connectivity.
- **Star/Radial** (133 nodes, 504 directed edges): A central hub connected to 12 ring nodes, which form a circle and each have 10 leaf nodes in chains. Cross-sector traffic must pass through the hub. Represents European-style cities with radial road patterns.
- **Barabási-Albert** (150 nodes, 592 directed edges): Generated by preferential attachment, producing a scale-free degree distribution. A few high-degree hub nodes have many connections, while most nodes have few. Represents cities with a few major arterial roads and many minor streets.

11.4 Results

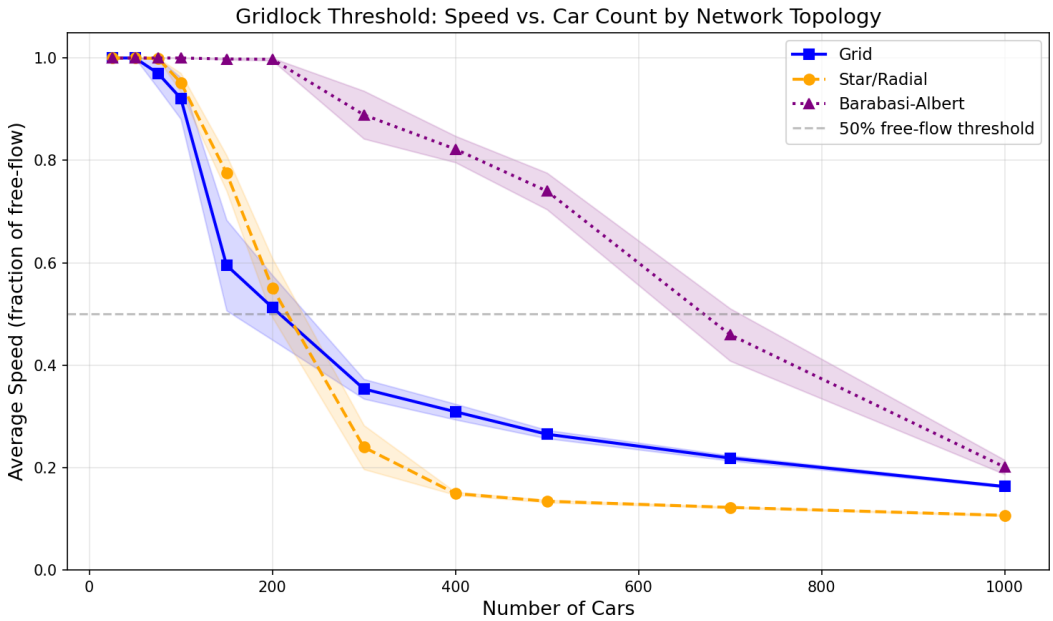

Figure 16: Average speed (as fraction of free-flow) vs. number of cars for each network topology. Shaded bands show ±1 standard error of the mean across 20 runs. All three topologies start near 1.0 at low car counts but diverge sharply as car count increases, each exhibiting a qualitatively different congestion profile.

The three curves in Figure 16 show three distinct congestion profiles:

Star/Radial (orange): The sharpest collapse. Speed drops almost vertically between 50 and 300 cars, falling from near-free-flow to below 0.2. This is the signature of a single bottleneck: once the central hub saturates, nearly all cross-sector traffic stalls simultaneously. There is no graceful degradation; the network is either flowing or gridlocked.

Grid (blue): A steep initial decline followed by a **plateau**. Speed falls to around 0.3–0.4 by 200 cars, but then stabilizes in that range until roughly 500 cars before continuing to decline. This plateau suggests partial gridlock: the central rows and columns (where shortest paths concentrate) are congested, but peripheral routes still flow. The grid is congested but not fully stopped, a qualitatively different regime from the star's all-or-nothing collapse.

Barabási-Albert (purple): The most gradual decline. Speed stays near 1.0 well past 200 cars and decreases steadily without a sharp cliff. The network degrades incrementally as progressively lower-degree hubs saturate one by one, rather than collapsing at a single critical point. Beyond approximately 700 cars, all three topologies converge to near-zero speeds.

The tight standard error of the mean bands confirm that these differences are reproducible across runs.

11.5 Discussion

The speed curves reveal three qualitatively distinct congestion regimes, each a consequence of topology:

- **Star/Radial: catastrophic collapse.** The hub is a single point of failure. Once it saturates, congestion is self-reinforcing (dense edges slow cars, which keeps them on the edge longer, which increases density further) and the entire network grinds to a halt. The speed curve shows a sharp boundary between the "flowing" and "gridlocked" regimes.

- **Grid: partial gridlock with a plateau.** Shortest-path routing concentrates traffic on central rows and columns, which congest at relatively low car counts. But the grid's many parallel paths allow peripheral routes to keep flowing, stabilizing average speed at a mediocre level. The grid's redundancy is only partially useful here because our simulation uses static routing; with dynamic rerouting, the grid's alternative paths would become more effective.

- **Barabási-Albert: gradual degradation.** The scale-free degree distribution means a few hubs have many edges and can absorb heavy traffic before saturating. As car count increases, hubs congest one by one in order of decreasing degree, producing a smooth decline rather than a sharp cliff. The many edges connecting low-degree peripheral nodes rarely appear on shortest paths and stay uncongested until very high car counts.

The same update rule on three different topologies produces three different congestion profiles: the grid plateaus, the star collapses, and the BA network slopes down gradually.

The BA network's resilience suggests a design principle for real cities: a road network with a few high-capacity arterial roads (hubs) connected by many smaller streets degrades more gracefully under load than either a uniform grid or a centralized radial layout. The hub roads absorb disproportionate traffic, and when one hub saturates, traffic redistributes to others rather than cascading into system-wide gridlock. Los Angeles exemplifies this pattern: its freeway system functions as a set of high-degree hubs (the 405, 101, 10, and 110 freeways) connecting a sprawling network of surface streets. Despite its reputation for traffic, the system handles enormous volumes precisely because congestion on one freeway does not immediately paralyze the rest of the network. Drivers have alternative arterials, and the load distributes across multiple high-capacity corridors. A city built around a single central boulevard (the star topology) would collapse far more readily under the same total traffic volume.

12 Conclusion

The original ChatGPT-generated simulation's binary threshold fails to capture how real traffic behaves. By implementing the Greenshields speed-density model as a cellular automaton

on a network graph, we obtain continuous congestion dynamics that can be meaningfully measured and visualized.

Edge betweenness centrality predicts empirical congestion density because cars follow shortest paths, and edges on many shortest paths carry more traffic. This relationship holds across both tested real-world network topologies (Spearman $r = 0.88$ on Corrientes, $r = 0.81$ on Palermo), suggesting it is a general property of shortest-path routing on road networks.

The gridlock analysis on synthetic networks demonstrates that network topology determines the qualitative character of congestion: the star/radial collapses catastrophically, the grid reaches a partially-congested plateau, and the scale-free BA network degrades gradually. The same Greenshields update rule produces three distinct congestion regimes depending on topology, showing that network topology, not the update rule, determines how congestion plays out.

Learning Outcomes

#cs110-CodeReadability. We applied this LO by fixing the six bugs in the original ChatGPT-generated notebook, restructuring the code into three documented classes (`RoadNetwork`, `Car`, `TrafficSimulation`) with docstrings, and adding inline comments explaining each part of the simulation logic (Section 2).

#modeling. We applied this LO by analyzing how the original binary threshold model works, identifying its structural limitations (binary congestion, arbitrary threshold, no position tracking), and redesigning the simulation using the Greenshields speed-density relationship framed as a cellular automaton on a network graph (Sections 3–6).

#dataviz. We applied this LO by critiquing the original visualizations (which show traffic volume, not congestion) and building improved visualizations: car animations showing Greenshields-derived speeds, density heatmap animations showing congestion evolving over time, and static average-density heatmaps that measure actual congestion in cars/meter (Section 7). We also designed side-by-side spatial comparison maps and bar charts to visually compare empirical congestion against network metrics (Sections 9 and 10).

#EmpiricalAnalysis. We applied this LO by running 50 independent simulations to identify structurally congested roads, performing a convergence analysis to justify the ensemble size, and repeating the analysis on a second network (Plaza Italia / Palermo) to validate our findings (Sections 8 and 10).

#TheoreticalAnalysis. We applied this LO by identifying edge betweenness centrality as a structural predictor of congestion, computing Spearman rank correlations on two networks ($r = 0.88$ and $r = 0.81$), and analyzing how three synthetic network topologies (grid, star/radial, Barabási-Albert) produce qualitatively different congestion regimes under the same update rule (Sections 9 and 11).

#composition and #organization. We applied these LOs by structuring the writeup to follow a logical progression: bug fixes, model critique, model improvement, new visualizations, empirical analysis, theoretical prediction, validation, and bonus topology analysis. Each section builds on the previous one, and figures are captioned with quantitative detail.

AI Use Statement

In this project, AI was only used to format and structure the code and written text. All analysis decisions, experiment design, and interpretation of results are our own.

References

Greenshields, B. D. (1935). A study of traffic capacity. In *Proceedings of the Highway Research Board*, volume 14, pages 448–477.

Nagel, K. and Schreckenberg, M. (1992). A cellular automaton model for freeway traffic. *Journal de Physique I*, 2(12):2221–2229.

Spearman, C. (1904). The proof and measurement of association between two things. *The American Journal of Psychology*, 15(1):72–101.